

[프로젝트 1] MiniJava 파서, AST 빌드, PrettyPrint

학과: 소프트웨어공학과

학번: 213652

이름: 김지환

목차

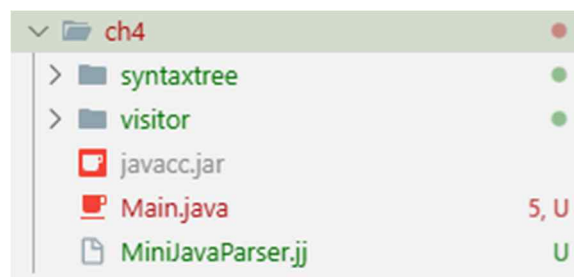
0. 파일 옮기기
1. 초기 빌드 테스트
2. MiniJavaParser.jj에 Semantic Actions 추가
3. 테스트
4. 고찰 및 회고

0. 파일 및 폴더 복사

ch4 폴더를 생성한 뒤, AndrewAppel/chap4/handcrafted 폴더에 있는 syntax tree 폴더, visitor 폴더, Main.java를 옮겼다.

이후 ch3에서 파싱만을 위해 만들어 두었던 MiniJavaParser.jj를 옮겼다.

JavaCC 환경 같은 경우 시스템 전역 환경 변수를 설정하는 방식 대신 프로젝트 디렉토리 내에 직접 javacc.jar를 포함시키는 방식을 채택하였다.



1. 초기 빌드 테스트

AST 변환 작업 전, `java -cp javacc.jar javacc MiniJavaParser.jj` 명령어를 통해 제대로 파서가 생성되는지 확인하였다.

```
PS C:\Users\user\minijava_compiler\ch4> java -cp javacc.jar javacc MiniJavaParser.jj
Java Compiler Compiler Version 7.0.13 (Parser Generator)
(type "javacc" with no arguments for help)
Reading from file MiniJavaParser.jj . . .
File "TokenMgrError.java" does not exist. Will create one.
File "ParseException.java" does not exist. Will create one.
File "Token.java" does not exist. Will create one.
File "JavaCharStream.java" does not exist. Will create one.
Parser generated successfully.
```

이후 `javac *.java` 명령어를 통해 전체 클래스 파일 컴파일을 시도하였지만 다음과 같이 에러가 발생하였다.

```
PS C:\Users\user\minijava_compiler\ch4> javac *.java
Main.java:7: error: incompatible types: void cannot be converted to Program
    Program root = new MiniJavaParser(System.in).Goal();
                                                ^
Note: Some input files use unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
1 error
```

이는 새롭게 적용된 Main.java의 드라이버 코드가 `Goal()` 함수의 파싱 결과물로 AST의 루트인 Program 객체를 반환할 것으로 기대하고 있지만, 3장에서 작성한 기존 `Goal()` 함수는 단순히 문법 검사만 수행하고 반환형을 가지지 않는 void 상태이기 때문에 발생한 에러이다.

2. MiniJavaParser.jj에 Semantic Actions 추가

1단계에서 확인된 컴파일러 타입 불일치 문제를 해결하기 위해, MiniJavaParser.jj 파일 전체를 AST 노드를 반환하는 구조로 재작성하였다.

2.1 패키지 연동

AST 구조 클래스들을 미리 모아둔 `syntaxtree` 패키지를 사용하기 위해 `PARSER_BEGIN` 구역에 import문을 통해 패키지 연동을 하였다.

```
5  PARSER_BEGIN(MiniJavaParser)
6
7  import syntaxtree.*;
8
9  public class MiniJavaParser {
10 }
11 PARSER_END(MiniJavaParser)
```

2.2 Identifier, Literal

가장 먼저 하위에 위치하는 리프 노드(식별자, IntegerLiteral 등)부터 AST 객체로 추출하는 리팩토링을 진행하였다. 어휘 분석기에서 얻은 Token t 에서 실제 값만을 추출한 뒤, `syntaxtree` 패키지에 정의된 Identifier, IntegerLiteral 등의 AST 노드로 변환하여 반환하도록 수정하였다. 단,

IntegerLiteral 생성자 호출 인자 타입은 String이 아닌 Integer이기 때문에 Integer.parseInt를 통해서 전달했다.

```
205 Exp IntegerLiteral() :
206 { Token t; }
207 {
208     t = <INTEGER_LITERAL>
209     { return new IntegerLiteral(Integer.parseInt(t.image)); }
210 }
```

2.3 Type

변수 선언 등에 사용되는 자료형 또한 AST 상에서 개별적인 노드로 취급하고 있기 때문에 반환형을 수정하였다.

int 나 boolean 같은 기본 자료형들은 각각 매칭되는 순수 객체를 바로 반환하도록 수정하였고, IdentifierType의 경우에는 Identifier 객체의 실제 이름 값을 통해 타입 객체로 만들어 반환하도록 하였다.

```
154 Type Type() :
155 { Type t; Identifier i; }
156 {
157     LOOKAHEAD(3) t=ArrayType() { return t; }
158     | t=BooleanType() { return t; }
159     | t=IntegerType() { return t; }
160     | i=Identifier() { return new IdentifierType(i.s); }
161 }
```

2.4 Primary Expression

괄호 표기식, this 지시자, 배열 및 객체 할당 구문이 포함된 PrimaryExpression 규칙의 반환형을 Exp로 수정하였다. 각 구문이 파싱될 때 This, NewArray, NewObject 등의 개별적인 AST 노드 객체를 생성하여 반환하도록 모두 수정해주었다.

```

189 Exp PrimaryExpression() :
190 { Exp e; Identifier i; }
191 {
192   (
193     | e=IntegerLiteral()
194     | e=TrueLiteral()
195     | e=FalseLiteral()
196     | i=Identifier() { e = new IdentifierExp(i.s); }
197     | e=ThisExpression()
198     | LOOKAHEAD(3) e=ArrayAllocationExpression()
199     | e=AllocationExpression()
200     | <LPAREN> e=Expression() <RPAREN>
201   )
202   { return e; }
203 }

```

2.5 Postfix Expression

메서드 호출, 배열 인덱스, 배열 길이 접근을 담당하는 PostfixExpression 규칙의 반환형을 Exp로 수정하였다. 파싱 시 대괄호([], 소괄호()) 기호는 제외하고, 내부의 ExpList나 배열 인덱스 등의 미 있는 부분만 활용하여 각 AST 노드의 생성자 인자로 전달하여 반환하도록 구현하였다.

메서드 호출 구문의 인자 리스트 처리 시엔 매개변수가 없는 경우의 Null 에러를 방지하기 위해 ExpList 객체를 기본값으로 먼저 할당해 두었다. 이후 EBNF 대괄호 [] 구문을 이용하여, 실제 내부 표현식 인자가 존재해 파싱될 때에만 해당 결과 객체로 덮어쓰워 전달되도록 구현하였다.

```

242 Exp PostfixExpression() :
243 { Exp e1, e2; Identifier i; ExpList el; }
244 {
245   e1=PrimaryExpression()
246   (
247     | <LBRACK> e2=Expression() <RBRACK> { e1 = new ArrayLookup(e1, e2); }
248     | LOOKAHEAD(2) <DOT> <LENGTH> { e1 = new ArrayLength(e1); }
249     | LOOKAHEAD( <DOT> Identifier() <LPAREN> ) <DOT> i=Identifier() <LPAREN>
250     { el = new ExpList(); } [ el=ExpressionList() ]
251     <RPAREN> { e1 = new Call(e1, i, el); }
252   )*
253   { return e1; }
254 }
255 }

```

2.6 Unary Expression

부정 연산을 지정하는 UnaryExpression 구문 규칙의 반환형을 Exp로 수정하였다. 파싱 과정에서 ! 기호를 읽었을 경우, 우측에 뒤따르는 e를 재귀적으로 파싱한 뒤 이를 Not AST 노드의 생성자 인자로 전달해 객체를 반환하도록 구현하였다.

```

277 Exp UnaryExpression() :
278 { Exp e; }
279 {
280     <BANG> e=UnaryExpression() { return new Not(e); }
281     | e=PostfixExpression() { return e; }
282 }

```

2.7 Binary Expression

사칙연산 및 논리 연산은 ()* 반복문으로 구현되어 있다. 연산자 우선순위와 좌결합 원칙이 반영 되도록 구현하기 위해 반복문 내에서 연산자가 인식될 때마다 기존의 좌측 자식 노드(e1)를 상위 연산 노드로 갱신하도록 하였다. 반복이 다 끝나게 되면 각 연산에 해당하는 객체를 반환하게 된다.

```

284 Exp MulExpression() :
285 { Exp e1, e2; }
286 {
287     e1=UnaryExpression() ( <STAR> e2=UnaryExpression() { e1 = new Times(e1, e2); } ) *
288     { return e1; }
289 }

```

2.8 Statement, Block

출력문(PrintStatement), 대입 구문(AssignmentStatement 등) 그리고 블록을 모두 Statement 반환 형으로 수정하였다. 출력문은 내부 표현식을 Print 노드로 감싸 객체를 반환하도록 하였으며, 대입 구문은 연산자(=) 좌측의 식별자와 우측의 계산식이 파싱된 결과 객체를 각각 변수(i, e)로 받은 뒤, 이를 Assign 계열 노드의 생성자 인자로 전달해 반환하도록 구현하였다.

```

363 Statement AssignmentStatement() :
364 { Identifier i; Exp e; }
365 {
366     i=Identifier() <ASSIGN> e=Expression() <SEMI>
367     { return new Assign(i, e); }
368 }

```

제어 명령문인 While, If의 경우 괄호 안의 조건식(Exp)과 실행 문장(Statement)을 각 Statement 생성자의 인자로 전달해 반환하도록 구현하였다.

```

342 Statement WhileStatement() :
343 { Exp e; Statement s; }
344 {
345     <WHILE> <LPAREN> e=Expression() <RPAREN> s=Statement()
346     { return new While(e, s); }
347 }

```

마지막으로 Block은 중괄호 {} 내부에서 문장이 반복 파싱될 때마다 StatementList에 요소를 추가 하고, 최종적으로 이 리스트 객체를 Block의 생성자의 인자로 전달해 반환하도록 구현하였다.

```

314 Statement Block() :
315 { StatementList sl = new StatementList(); Statement s; }
316 {
317     <LBRACE> ( s=Statement() { sl.addElement(s); } ) * <RBRACE>
318     { return new Block(sl); }
319 }

```

2.9 Declaration

ClassDeclaration, MethodDeclaration, VarDeclaration 및 매개변수 구문들의 반환형을 각각의 특화된 AST 노드 객체로 수정하였다. 파싱 과정 중 클래스명, 상속 관계, 변수 리스트와 메서드 리스트 등을 각 노드의 생성자 인자로 전달해 객체를 반환하도록 구현하였다.

```

107 ClassDecl ClassExtendsDeclaration() :
108 { Identifier i, j; VarDeclList vl = new VarDeclList(); VarDecl v; MethodDeclList ml = new MethodDeclList(); MethodDecl m; }
109 {
110     <CLASS> i=Identifier() <EXTENDS> j=Identifier() <LBRACE>
111     ( v=VarDeclaration() { vl.addElement(v); } ) *
112     ( m=MethodDeclaration() { ml.addElement(m); } ) *
113     <RBRACE>
114     { return new ClassDeclExtends(i, j, vl, ml); }
115 }

```

2.10 Goal, Program

MainClass와 Goal이 각각 MainClass 및 Program AST 노드를 반환하도록 수정하였다. MainClass는 두 개의 식별자(클래스명, 메인 메서드 매개변수명)와 내부 실행 문장(Statement)을 변수로 받아 MainClass 노드의 생성자 인자로 넘겨주었다. Goal은 하위에서 반환된 MainClass와 모든 Declaration 노드들을 인자로 넘겨받아 단 하나의 Program 루트 노드로 묶어 반환하도록 하였다.

```

67 Program Goal() :
68 { MainClass m; ClassDeclList cl = new ClassDeclList(); ClassDecl c; }
69 {
70     m=MainClass()
71     ( c=TypeDeclaration() { cl.addElement(c); } ) *
72     <EOF>
73     { return new Program(m, cl); }
74 }

```

이를 최종적으로 Main.java 가 호출하여 전체 구조가 담긴 Program 객체를 반환 받게 되며, 이후 해당 객체를 통해 PrettyPrintVisitor 등의 방문자 알고리즘을 정상적으로 수행할 수 있게 된다.

3. 결과

3.1 JavaCC 파서 생성

다음과 같이 파서 생성이 정상적으로 된 것을 확인할 수 있었다.


```
PS C:\Users\user\minijava_compiler\ch4> java -cp javacc.jar javacc MiniJavaParser.jj
Java Compiler Compiler Version 7.0.13 (Parser Generator)
(type "javacc" with no arguments for help)
Reading from file MiniJavaParser.jj . . .
File "TokenMgrError.java" does not exist. Will create one.
File "ParseException.java" does not exist. Will create one.
File "Token.java" does not exist. Will create one.
File "JavaCharStream.java" does not exist. Will create one.
Parser generated successfully.
```

3.2 전체 Java 클래스 컴파일 결과

다음과 같이 에러 없이 정상적으로 빌드 되는 것을 확인할 수 있었다.

```
PS C:\Users\user\minijava_compiler\ch4> javac *.java
PS C:\Users\user\minijava_compiler\ch4> |
```

3.3 전체 프로그램 코드 파싱 및 Pretty Print 테스트

Get-ChildItem ../programs/*.java | ForEach-Object { echo "`n--- Testing \$_ ---"; cat \$_.FullName | java Main } 명령어를 통해 전체 프로그램에 대해 테스트를 진행했고 다음과 같이 8개 파일 모두 정상적으로 출력되는 것을 확인할 수 있었다.

```
PS C:\Users\user\minijava_compiler\ch4> Get-ChildItem ../programs/*.java | ForEach-Object { echo "`n--- Testing $_ ---"; cat $_.FullName | java Main }

--- Testing C:\Users\user\minijava_compiler\programs\BinarySearch.java ---
class BinarySearch {
    public static void main (String [] a) {
        System.out.println(new BS().Start(20)); }
}

class BS {
    int [] number;
    int size;
    public int Start (int sz) {
        int aux01;
        int aux02;
        aux01 = this.Init(sz);
        aux02 = this.Print();
        if (this.Search(8))
            System.out.println(1);
        else System.out.println(0);
        if (this.Search(19))
            System.out.println(1);
        else System.out.println(0);
    }
}
```

3.4 특정 프로그램 코드 파싱 및 Pretty Print 테스트

cat ../programs/Factorial.java | java Main 명령어를 통해 Factorial.java의 Pretty Print 테스트를 진행하였고 정상적으로 출력되는 것을 확인할 수 있었다.

```
PS C:\Users\user\minijava_compiler\ch4> cat ../programs/Factorial.java | java Main
class Factorial {
    public static void main (String [] a) {
        System.out.println(new Fac().ComputeFac(10)); }
}

class Fac {

    public int ComputeFac (int num) {
        int num_aux;
        if ((num < 1))
            num_aux = 1;
        else num_aux = (num * this.ComputeFac((num - 1)));
        return num_aux;
    }
}
```

출력 결과를 살펴보았을 때 Factorial.java 실제 코드와 완벽히 같진 않았다.

첫번째 차이점은 원본 코드의 class Factorial{ 이나 num_aux = 1 ; 와 같이 임의로 된 공백이나 띄어쓰기가 되지 않은 곳이 출력된 코드에선 일정한 띄어쓰기와 들여쓰기 등이 적용되어 있었다.

```
1 class Factorial{
    Run | Debug
2     public static void main(String[] a){
3         System.out.println(new Fac().ComputeFac(10));
4     }
5 }
```

```
class Factorial {
    public static void main (String [] a) {
        System.out.println(new Fac().ComputeFac(10)); }
}
```

두번째 차이점은 실제 소스 코드는 if (num < 1) 로 작성되었으나, 출력된 코드는 if ((num < 1)) 과 같이 이중 괄호로 겹쳐져 있었다. 또한 산술 연산에서도 (num * this.ComputeFac((num - 1))) 와 같이 각 연산마다 괄호가 감싸져 있었다.

```
int num_aux ;
if (num < 1)
|     num_aux = 1 ;
else
|     num_aux = num * (this.ComputeFac(num-1))
return num_aux ;
```

```
int num_aux;
if ((num < 1))
num_aux = 1;
else num_aux = (num * this.ComputeFac((num - 1)));
return num_aux;
```

다음으로 Factorial.java 에 세미콜론을 삭제하여 테스트를 진행했다.


```

1  class Factorial{
    Run | Debug
2      public static void main(String[] a){
3          System.out.println(new Fac().ComputeFac(10));
4      }
5  }
6
7  class Fac {
8
9      public int ComputeFac(int num){
10         int num_aux
11         if (num < 1)
12             num_aux = 1 ;
13         else
14             num_aux = num * (this.ComputeFac(num-1)) ;
15         return num_aux ;
16     }

```

다음과 같이 정상적으로 파싱 에러가 출력된 것을 확인할 수 있었다.

```

PS C:\Users\user\minijava_compiler\ch4> cat ../programs/Factorial.java | java Main
ParseException: Encountered "if" "if "" at line 11, column 2.
Was expecting:
    ";" ...

```

4. 고찰 및 회고

이번 과제는 미리 제공된 syntaxtree 패키지를 가져다 파서 구조에 단순히 연결해 주는 형태였기에 큰 어려움은 없었다. 하지만 만약 실제 컴파일러 개발자가 되어 이러한 AST 노드들을 밑바닥부터 직접 설계해야만 한다면, 수많은 코드 요소를 어느 수준까지 개별 노드로 세분화할지, 트리 객체 간의 상속 구조는 어떻게 엮어내야 할지 등 훨씬 더 복잡한 문제들을 깊이 고민해야 함을 깨달았다.

또한 파싱된 소스코드를 PrettyPrintVisitor로 다시 출력해 본 결과, 원본과 달리 불규칙한 공백은 사라지고 연산자마다 방어 괄호가 겹겹이 쳐져 나오는 차이점을 확인할 수 있었다. 이를 통해 컴파일러는 소스코드를 트리 노드 객체들로 해체하여 이해하고 동작한다는 사실을 확실하게 체감할 수 있었다.